

User Mode , Kernel Mode and System Calls

A Detailed Example: Write() to Console

```
write(1, "Hello", 5); // User-space C library call
```

After execution of above program what Happens Next:

Hardware Enforces Protection:

```
// CPU automatically does:
if (current_mode == USER_MODE) {
    if (instruction == "IN/OUT" || // I/O port access
        memory_address >= USER_SPACE_LIMIT) {
        GENERATE_FAULT; // Hardware exception!
    }
}
```

User mode restrictions:

- I/O instructions (IN/OUT) are prohibited → prevents user programs from directly accessing hardware.
- Memory access beyond user-space limit is prohibited → prevents accessing kernel memory or other processes' memory.
- If violated → CPU triggers a hardware exception (like General Protection Fault or Segmentation Fault), transferring control to the OS kernel.

Purpose:

Maintain system security and stability by preventing unprivileged user programs from interfering with hardware or kernel memory. Only the OS kernel (running in kernel mode) can perform these privileged operations.

Kernel Takes Over:

```
// Kernel validates everything:
if (!is_valid_user_pointer(buffer, 5)) {
    return -EFAULT; // Bad address error
}
if (!can_user_write_to_fd(fd)) {
    return -EPERM; // Permission error
}
// Kernel accesses hardware directly:
void write_to_tty(char *data, int len) {
    // Direct hardware access - user can't do this!
    outb(TTY_DATA_PORT, data[0]); // Write to I/O port
    outb(TTY_STATUS_PORT, TTY_WRITE); // Signal device

    // Or memory-mapped I/O:
    *((volatile uint32_t*)VIDEO_MEMORY) = data;
}
```

This shows kernel's role in mediating hardware access after verifying user requests:

Validation phase (safety checks):

- Checks if the user's buffer pointer is valid and accessible

- Verifies file descriptor (fd) permissions before proceeding
- Direct hardware access (privileged operations):
- The kernel can directly write to I/O ports (outb())
- Can access memory-mapped I/O (like video memory)
- These operations would cause a hardware fault if attempted from user mode

Key concept: The kernel acts as a trusted gatekeeper:

Validates all user requests for safety

- Performs the actual privileged hardware operations on the user's behalf
- Returns errors (-EFAULT, -EPERM) if requests are invalid
- This separation enables secure system calls: user programs request services, kernel validates and executes them safely.

Concrete Hardware Access Examples:

User program CANNOT do this:

```
void user_read_disk() {  
    // These are privileged instructions  
    outb(0x1F6, 0xE0 | (lba >> 24)); // I/O port access  
    outb(0x1F2, sector_count);        // ERROR: Program crashes!  
}
```

User code (user_read_disk)

Runs in user mode (Ring 3).

- Tries to execute privileged I/O instructions like outb (writing to hardware I/O ports).
- Modern CPUs and OSes forbid direct hardware access from user programs for safety and isolation.
- Result: the program crashes or triggers a protection fault because only the kernel may access disk controller ports.

Only kernel can do:

```
void kernel_read_disk(struct file *file, void *buf, size_t count) {  
    // 1. Program disk controller  
    outb(DISK_COMMAND_PORT, READ_COMMAND); // OK in Ring 0  
    // 2. DMA setup (direct memory access)  
    setup_dma_transfer(buf, count); // Programs hardware DMA  
    // 3. Wait for interrupt  
    wait_for_disk_interrupt();  
}
```

Kernel code (kernel_read_disk)

Runs in kernel mode (Ring 0).

Has permission to execute privileged instructions.

Safely:

- Programs the disk controller via I/O ports.
- Sets up DMA so the disk can transfer data directly to memory.
- Waits for a hardware interrupt signalling completion.
- This is how operating systems securely mediate hardware access for user programs.

Another Example

Memory Allocation (malloc() → brk()/mmap())

User view:

```
void *p = malloc(4096); // "Give me memory"
```

What actually happens:

1. `malloc()` calls `brk()` system call
2. Kernel updates process memory map
3. Kernel programs MMU (Memory Management Unit):
 `set_page_table_entry(process_cr3, virtual_addr, physical_addr);`
4. Hardware (MMU) enforces this mapping

Another Example:

Network Communication (send() system call)

User program:

```
send(socket, data, len, 0); // Request to send data
```

Kernel handles:

```
void kernel_send_packet(struct socket *sock, void *data, int len) {  
    // 1. Copy data from user to kernel buffer  
    copy_from_user(kernel_buf, data, len);  
  
    // 2. Program network card DMA  
    nic_setup_transmit_descriptor(kernel_buf, len);  
  
    // 3. Signal network card  
    outl(NIC_COMMAND_REGISTER, TRANSMIT_COMMAND);  
  
    // 4. Handle interrupts when packet sent  
}
```

What Happens If User Tries Direct Access

Attempt 1: Direct I/O Port Access

```
// user_program.c
#include <sys/io.h>

int main() {
    // Request I/O permission (requires root!)
    ioperm(0x60, 5, 1);

    // Try to read keyboard controller
    unsigned char data = inb(0x60);

    // On most systems: SEGFAULT or permission denied
    // Even with ioperm, modern OSes restrict this
}
```

System Call Sequence Example: File Copy Program (cp in.txt out.txt)

Suppose we have a program, to copy text from one from to another file

Step-by-Step Breakdown

Phase 1: Getting File Names

Option A: Command-line arguments (Non-interactive)

bash

cp source.txt destination.txt

System calls: Minimal (shell handles parsing)

User interaction: None

Option B: Interactive prompts (Interactive program)

text

Enter source filename: source.txt

Enter destination filename: destination.txt

System calls required:

write() - Display "Enter source filename:"

read() - Read user input for source file

write() - Display "Enter destination filename:"

read() - Read user input for destination file

Phase 2: Opening Files with Error Handling

Step 1: Open Input File

// User program requests:

```
open("source.txt", O_RDONLY);
```

// What kernel does:

- Checks if file exists
- Verifies read permissions
- Returns file descriptor or error

Possible errors:

- File doesn't exist → ENOENT error
- Permission denied → EACCES error

User program must handle:

```
int in_fd = open(source_file, O_RDONLY);
if (in_fd == -1) {
    write(2, "Error: Cannot open source file\n", 31);
    exit(1); // Another system call
}
```

Step 2: Create/Open Output File

```
// User program requests:
open("destination.txt", O_WRONLY | O_CREAT | O_EXCL, 0644);
// O_EXCL ensures file doesn't exist
```

Possible scenarios:

- File doesn't exist: Creates new file
- File exists: Returns error (or asks user)
- Permission issues: Returns error

Phase 3: Copying Data

Step 3: Read-Write Loop

```
char buffer[BUFFER_SIZE];
ssize_t bytes_read;

while ((bytes_read = read(in_fd, buffer, BUFFER_SIZE)) > 0) {
    // Kernel actions during read():
}
```

```

// 1. Checks file descriptor validity
// 2. Accesses disk hardware (if not cached)
// 3. Copies data to kernel buffer
// 4. Copies from kernel to user buffer

write(out_fd, buffer, bytes_read);
// Kernel actions during write():
// 1. Validates buffer
// 2. Copies to kernel buffer
// 3. Eventually writes to disk
}

// End of file reached when read() returns 0
// Error occurs when read() returns -1

```

Error conditions during copy:

Read errors: Disk failure, bad sector

Write errors: Disk full, permission change

Hardware errors: Network disconnection (if remote)

Phase 4: Cleanup and Completion

Step 4: Close Files

- `close(in_fd);` // System call to release resources
- `close(out_fd);` // Flushes any buffered data to disk
- What `close()` does:
 - Flushes pending writes to disk
 - Frees kernel data structures
 - Releases file descriptor for reuse

Step 5: Display Completion Message

```
write(1, "File copied successfully\n", 25);
```

Step 6: Terminate Program

```
exit(0); // Exit with success status
```

	User Program Flow	System Calls Used
1	Get source filename →	<code>write()</code> , <code>read()</code> [if interactive]

2	Get dest filename →	write(), read() [if interactive]
3	Open source file → └─ File doesn't exist? → └─ Permission denied? →	open() write(), exit() write(), exit()
4	Create dest file └─ File exists? → └─ Permission denied? →	open() with O_CREAT write(), read(), unlink(), open() write(), exit()
5	Copy loop: Read chunk → └─ Disk error? → └─ EOF reached? Write chunk → Disk full? →	read() write(), exit() break loop write() write(), exit()
6	Close files →	close(), close()
7	Show completion →	write()
8	Exit program	→ exit()